NegMath: Operator-Inversion Diagnostics for Complex Mathematical Systems

Abstract

NegMath is introduced as a lightweight **operator-inversion diagnostic** for complex algebraic, dynamical, and PDE systems. Rather than defining a new algebra, NegMath applies a deliberately non-homomorphic transform $\mathcal{T}$ that flips signs and dualizes operators (addition to subtraction, multiplication to division, derivatives to integrals) in selected sub-expressions. The resulting "shadow" equations are not meant to be physically correct; instead, the way they **fail**—loss of commutativity, explosion of condition numbers, emergence of stable zeros or structured NaNs—exposes **anchors** (dominant terms), **stress regions**, and topological structure in the original system. This paper formalizes NegMath as a reproducible diagnostic, presents two concrete worked examples with code and plots (1D viscous Burgers and Lotka–Volterra dynamics), and sketches speculative applications in information theory and cryptography.

1. Introduction

Classical modeling often runs into breakdowns: Navier–Stokes at high Reynolds numbers, stiff nonlinear ODEs, or high-dimensional algebras in coding theory. Traditional tools—adjoint methods, perturbation expansions, spectral analysis—are powerful but heavy. NegMath takes the opposite approach: a **crude, fast, intentionally**

destructive transform that can be implemented as a one-line symbolic rewrite or a small wrapper around existing code. The intuition is simple: *invert* the local algebra, then observe what survives and where the system breaks first.

NegMath is not a new number system. It is a **probe**: given an expression E built from familiar operators, a local map $\mathcal{T}$ constructs a shadow expression $\mathcal{T}(E)$ that inverts selected operators. Running the original and shadow systems in parallel reveals structural features: which terms behave like topological anchors, which regions in space-time carry the "debt" of singularities, and where numerical solvers are about to fail.

2. NegMath as a local operator-inversion probe

2.1 Definition on algebraic expressions

Let E be an expression over scalars, variables, and binary operators $+, -, \times, \div$. The **NegMath probe** $\mathcal{T}$ is a local transform defined on a chosen subexpression $E_{\text{sub}} \subset E$:

- Atoms: for any scalar or variable a, $\mathcal{T}(a) = -a$.
- Addition: for a sum $a + b + \ldots$,
$$\mathcal{T}(a + b + \ldots) = \mathcal{T}(a) - (\mathcal{T}(b) + \ldots).$$
- Multiplication: for a product $abc\ldots$,
$$\mathcal{T}(abc\ldots) = \frac{\mathcal{T}(a)}{\mathcal{T}(b)\,\mathcal{T}(c)\ldots}.$$
- Powers and derivatives are extended heuristically when needed (e.g. $\mathcal{T}(x^n)$ as a root of $\mathcal{T}(x)$, or $\mathcal{T}(\partial_x f)$ as an integral of $\mathcal{T}(f)$).

The key point: $\mathcal{T}$ is **not** a homomorphism and is not applied globally. It is a controlled, localized corruption applied to specific subterms.

2.2 Diagnostic outputs

Given a system of equations $F(u) = 0$ or $\dot{u} = f(u)$, NegMath is used as follows:

1. Select a term or block T_i in the equations.
2. Construct a shadow system where T_i is replaced by $\mathcal{T}(T_i)$ while leaving all other terms intact.
3. Compare behavior of the original and shadow systems under the same inputs.

From this, several diagnostics can be defined:

- **Anchor score** for term T_i: a scalar measuring how much the solution changes when only T_i is probed (e.g., RMS distance between trajectories).
- **Topological index**: a qualitative indicator of how often commutativity/associativity is destroyed in $\mathcal{T}(E)$ while the system as a whole remains interpretable.
- **Shadow zero / NaN patterns**: spatial or temporal regions where the shadow system produces small magnitudes, zeros, or structured NaNs while the original remains finite.

These are not theorems; they are **empirical signals** computed from side-by-side experiments.

3. Example I: viscous Burgers equation

3.1 Setup

Consider the 1D viscous Burgers equation on $[0, 1]$ with periodic boundary conditions:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2},$$

with viscosity $\nu = 0.01$ and initial condition $u(x, 0) = \sin(2\pi x)$. The nonlinear advection term uu_x is the main source of steep gradients and potential shock formation.

NegMath is used here as a **pointwise diagnostic** on the advective term. After numerically evolving $u(x, t)$ to some time T, a simple

shadow quantity is computed:

$$D(x,T) = \frac{u(x,T)}{u_x(x,T) + \varepsilon},$$

with a small $\varepsilon > 0$ for regularization. This corresponds to taking the original product $u \cdot u_x$ and applying $\mathcal{T}(ab) \approx a/b$ up to sign conventions.

3.2 Numerical experiment

A standard finite-difference scheme with periodic boundaries is used (central differences for u_x and u_{xx}, explicit Euler in time). The code below reproduces the experiment and writes both the raw data and plots suitable for a repository.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Domain and discretization
L = 1.0
Nx = 200
x = np.linspace(0, L, Nx, endpoint=False)
dx = L / Nx

nu = 0.01
T_final = 0.5
dt = 2e-4
nsteps = int(T_final / dt)

# Initial condition
u = np.sin(2 * np.pi * x)

# Time stepping (periodic BCs)
for _ in range(nsteps):
    u_ip1 = np.roll(u, -1)
    u_im1 = np.roll(u, 1)
    ux = (u_ip1 - u_im1) / (2 * dx)
```

```
    uxx = (u_ip1 - 2 * u + u_im1) / dx**2
    u = u + dt * (-u * ux + nu * uxx)

# Final diagnostics
u_ip1 = np.roll(u, -1)
u_im1 = np.roll(u, 1)
ux = (u_ip1 - u_im1) / (2 * dx)

eps = 1e-3
D = u / (ux + eps)

# Save CSV for analysis
burgers_df = pd.DataFrame({"x": x, "u": u, "ux": ux, "D": D})
burgers_df.to_csv("burgers_negmath_diagnostic.csv", index=False)

# Plot field and gradient
plt.figure(figsize=(6, 4))
plt.plot(x, u, label="u(x, T)")
plt.plot(x, ux, label="u_x(x, T)")
plt.xlabel("x")
plt.ylabel("u, u_x")
plt.legend()
plt.title("Viscous Burgers: field and gradient at T=%.2f" % T_final)
plt.tight_layout()
plt.savefig("burgers_field_gradient.png", dpi=150)
plt.close()

# Plot NegMath diagnostic
plt.figure(figsize=(6, 4))
plt.plot(x, np.abs(D), label="|u/(u_x+eps)|")
plt.xlabel("x")
plt.ylabel("|D|")
plt.title("NegMath-style diagnostic for Burgers")
plt.legend()
plt.tight_layout()
plt.savefig("burgers_diagnostic.png", dpi=150)
plt.close()
```

3.3 Observations

- The Burgers field $u(x, T)$ and its gradient $u_x(x, T)$ are smooth at $T = 0.5$ but already show emerging steep regions where gradients amplify.
- The diagnostic $|D(x, T)| = |u/(u_x + \varepsilon)|$ spikes near locations where u_x is small or changes sign while u remains moderate. These are precisely the regions where the advective term uu_x is most sensitive to perturbations and where finite-difference solvers tend to lose accuracy first.
- In practice, D can be used as a **cheap indicator field**: cells with large $|D|$ suggest a need for mesh refinement, local time-step reduction, or more careful flux limiting, even before the primary solver exhibits visible instability.

4. Example II: anchor detection in Lotka–Volterra dynamics

4.1 Setup

Consider the classical Lotka–Volterra predator–prey system:

$$\dot{x} = \alpha x - \beta xy,$$
$$\dot{y} = \delta xy - \gamma y,$$

with $\alpha, \beta, \delta, \gamma > 0$. Here, x is prey, y is predator, and the four terms correspond to growth, interaction, and decay processes.

NegMath is used here as an **anchor detector**: each term is probed in isolation by a local operator inversion (multiplication to division, variable to its reciprocal), and the resulting trajectories are compared with the baseline.

4.2 NegMath term probes

A simple heuristic consistent with the multiplicative rule $\mathcal{T}(ab) = \mathcal{T}(a)/\mathcal{T}(b)$ leads to transforms like $\alpha x \mapsto \alpha/x$ and $\beta xy \mapsto \beta/(xy)$. With a small regularization ε to avoid division by zero, define

- $T_1(x) = \alpha/(x + \varepsilon)$ for the prey growth term.

- $T_2(x, y) = \beta/(xy + \varepsilon)$ for the interaction in $\dot{x}$.
- $T_3(x, y) = \delta/(xy + \varepsilon)$ for the interaction in $\dot{y}$.
- $T_4(y) = \gamma/(y + \varepsilon)$ for the predator decay term.

Four probed systems are then defined, each replacing only one term by its T_i version and leaving all others unchanged.

4.3 Numerical experiment and anchor scores

With parameters $\alpha = 1, \beta = 0.5, \delta = 0.5, \gamma = 2$ and initial condition $(x_0, y_0) = (2, 1)$, the baseline and probed systems are integrated using a fourth-order Runge–Kutta method over a fixed time horizon. The Python code below implements this experiment and computes **RMS distances** between the baseline trajectory and each probed trajectory as simple anchor scores.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

alpha, beta, delta, gamma = 1.0, 0.5, 0.5, 2.0
eps_anchor = 1e-3

# Baseline RHS
def lotka_rhs(x, y):
    dx = alpha * x - beta * x * y
    dy = delta * x * y - gamma * y
    return dx, dy

# NegMath-style term transforms
def T_term_alpha_x(x):
    return alpha / (x + eps_anchor)

def T_term_beta_xy(x, y):
    return beta / (x * y + eps_anchor)

def T_term_delta_xy(x, y):
    return delta / (x * y + eps_anchor)
```

```python
def T_term_gamma_y(y):
    return gamma / (y + eps_anchor)

# Probed systems
def lotka_rhs_probe_term1(x, y):
    dx = T_term_alpha_x(x) - beta * x * y
    dy = delta * x * y - gamma * y
    return dx, dy

def lotka_rhs_probe_term2(x, y):
    dx = alpha * x - T_term_beta_xy(x, y)
    dy = delta * x * y - gamma * y
    return dx, dy

def lotka_rhs_probe_term3(x, y):
    dx = alpha * x - beta * x * y
    dy = T_term_delta_xy(x, y) - gamma * y
    return dx, dy

def lotka_rhs_probe_term4(x, y):
    dx = alpha * x - beta * x * y
    dy = delta * x * y - T_term_gamma_y(y)
    return dx, dy

# RK4 integrator
def integrate_system(rhs, x0, y0, dt, nsteps):
    xs = np.empty(nsteps + 1)
    ys = np.empty(nsteps + 1)
    xs, ys = x0, y0
    x, y = x0, y0
    for n in range(nsteps):
        k1x, k1y = rhs(x, y)
        k2x, k2y = rhs(x + 0.5 * dt * k1x, y + 0.5 * dt * k1y)
        k3x, k3y = rhs(x + 0.5 * dt * k2x, y + 0.5 * dt * k2y)
        k4x, k4y = rhs(x + dt * k3x, y + dt * k3y)
        x += (dt / 6.0) * (k1x + 2 * k2x + 2 * k3x + k4x)
        y += (dt / 6.0) * (k1y + 2 * k2y + 2 * k3y + k4y)
        xs[n + 1], ys[n + 1] = x, y
```

```
    return xs, ys

x0, y0 = 2.0, 1.0
T_lv = 20.0
dt_lv = 0.01
nsteps_lv = int(T_lv / dt_lv)

traj_names = [
    "baseline",
    "probe_alpha_x",
    "probe_beta_xy",
    "probe_delta_xy",
    "probe_gamma_y",
]
rhs_fns = [
    lotka_rhs,
    lotka_rhs_probe_term1,
    lotka_rhs_probe_term2,
    lotka_rhs_probe_term3,
    lotka_rhs_probe_term4,
]

trajectories = {}
for name, rhs in zip(traj_names, rhs_fns):
    xs, ys = integrate_system(rhs, x0, y0, dt_lv, nsteps_lv)
    trajectories[name] = (xs, ys)

# Anchor scores: RMS distance vs baseline
base_x, base_y = trajectories["baseline"]
anchor_scores = {}
for name in traj_names[1:]:
    xs, ys = trajectories[name]
    dist = np.sqrt((xs - base_x) ** 2 + (ys - base_y) ** 2)
    rms = np.sqrt(np.mean(dist ** 2))
    anchor_scores[name] = rms

# Save trajectories and scores
n = nsteps_lv + 1
```

```
time_lv = np.linspace(0, T_lv, n)

lv_data = {"t": time_lv}
for name in traj_names:
    xs, ys = trajectories[name]
    lv_data[f"x_{name}"] = xs
    lv_data[f"y_{name}"] = ys

lv_df = pd.DataFrame(lv_data)
lv_df.to_csv("lotka_negmath_trajectories.csv", index=False)

scores_df = pd.DataFrame({"term": list(anchor_scores.keys()),
                          "rms_distance": list(anchor_scores.values())})
scores_df.to_csv("lotka_anchor_scores.csv", index=False)

# Phase-plane plot for the most disruptive term
most_disruptive = max(anchor_scores.items(), key=lambda kv: kv[1])

plt.figure(figsize=(6, 4))
plt.plot(base_x, base_y, label="baseline", linewidth=2)
plt.plot(trajectories[most_disruptive], trajectories[most_disruptive][1],
         label=most_disruptive, linewidth=1)
plt.xlabel("x (prey)")
plt.ylabel("y (predator)")
plt.title("Lotka–Volterra: NegMath term probe vs baseline")
plt.legend()
plt.tight_layout()
plt.savefig("lotka_phase_anchor.png", dpi=150)
plt.close()
```

4.4 Observations

- Each term probe induces a different deformation of the phase portrait. Terms whose NegMath inversion causes a large RMS trajectory deviation are good candidates for **anchors**: they control the qualitative behavior of the dynamics.
- In numerical experiments with the parameters above, the interaction terms often emerge as the strongest anchors, while

transforming pure growth/decay terms may primarily rescale trajectories without changing their topology.
- The method generalizes: for any nonlinear dynamical system, one can loop over terms, apply local NegMath probes, and rank terms by their anchor scores as a fast, model-agnostic sensitivity analysis.

5. Speculative applications beyond toy models

5.1 Information theory and coding

High-dimensional algebras and polynomial maps appear throughout coding theory and cryptography (multivariate quadratic systems, LDPC codes, lattice-based schemes). Such systems often have large **degenerate subspaces** that carry redundancy.

NegMath suggests a generic way to expose degeneracy:

- Apply $\mathcal{T}$ to selected products and sums in the encoding or decoding equations.
- Track how quickly the shadow system collapses to low-rank or zero subspaces under composition.
- Terms or directions that are insensitive under repeated NegMath probing likely correspond to redundant or compressible structure, while highly sensitive directions carry genuine information content.

This is closely aligned with the idea of using non-associativity and zero-divisor structure as resources for coding and obfuscation.

5.2 Cryptographic hardness and algebraic chaos

In noncommutative or nonassociative cryptographic primitives (e.g. certain loop-based designs), security relies partly on the difficulty of reconstructing hidden operations from observable algebraic behavior. NegMath-style probes could be used in two opposite ways:

- **Defender perspective:** design operations such that NegMath probes of public equations produce apparent "algebraic noise"—

no stable anchors, rapid loss of associativity—making structural discovery harder.
- **Attacker perspective:** use NegMath to attempt to **tease out anchors** in a candidate scheme by probing one term at a time and looking for consistent structural deformation, potentially revealing hidden linearity or exploitable subspaces.

At present this is speculative; rigorous hardness results would require careful analysis. But NegMath supplies a unifying language and a set of quick experiments to explore such ideas before committing to full-scale proofs.

5.3 Graph geometry and pathfinding

Weighted graphs are typically studied in the min-plus semiring. A NegMath-style dual can be formed by negating or reciprocating edge weights and altering the combination rule (sum to difference, product to quotient). Running standard shortest-path algorithms on this shadow graph yields alternative notions of "distance" that emphasize bottlenecks or topological shortcuts.

For example:

- In hierarchical or hyperbolic graphs, the NegMath shadow may highlight paths that trade off length for hierarchical depth, approximating certain log-norm distances.
- Anchor detection in this context amounts to asking which edges, when probed, most strongly perturb the overall distance matrix—a proxy for identifying critical links or communities.

6. Limitations and best practices

NegMath is intentionally **not** a rigorous algebraic framework. Its power and its risk come from the same source: $\mathcal{T}$ is a flexible, user-defined transform.

Best practices include:

- Always specify the exact variant of $\mathcal{T}$ used in an experiment so results are reproducible.

- Interpret NegMath outputs as **diagnostics**, not as new physical laws or proofs.
- Combine NegMath rankings (anchors, stress regions) with more traditional sensitivity analysis or adjoint methods before drawing strong conclusions.

In low-entropy, linear systems, NegMath will often add little; its real value is in nonlinear, high-chaos, or poorly understood regimes where a quick structural MRI is more valuable than exact formulas.

7. Conclusion

NegMath reframes algebraic breakdown as a signal rather than a failure. By systematically inverting local operators and watching what collapses first, it reveals hidden anchors, stress regions, and topological features in complex models. The Burgers and Lotka–Volterra examples illustrate how simple code and plots can turn this idea into a practical tool. Extending the same philosophy to PDEs, graph algorithms, coding theory, and cryptography may open up a new family of fast, exploratory diagnostics that complement existing high-precision methods.

References

1. negmath02.txt - **NegMath ($\mathcal{M}^{-}$): A Lightweight Operator-Inversion Heuristic for Structural Diagnosis, ...